

Bem-vindos à Debug Arena

Uma oficina para investigar bugs, testar hipóteses e defender soluções.

Equipe

André Luiz Ribeiro

Douglas Araújo Silva

Luiz Guyllherme Rodrigues

Nathanael Noberto da Silva

Thuany Guimarães Pereira

Disciplina

Inovações Educacionais
em Computação (Pós-
Graduação)

Data

03/06/2026, 13:00h -
17:00h



O que vamos aprender hoje?



Reconhecer erros

Identificar tipos de erros em programação: sintaxe, lógica e execução.



Resolver em pares

Colaborar, discutir hipóteses e chegar juntos à solução.



Comparar abordagens

Entender as diferenças entre depuração manual e com suporte de IA.



Aplicar estratégias

Usar técnicas de debugging de forma estruturada e investigativa.



Usar IA com crítica

Aproveitar a IA como apoio investigativo, sem pedir código pronto.



Defender soluções

Argumentar tecnicamente sobre a solução encontrada para a turma.

Nossa missão em 4 ciclos

1º Ciclo

Introdução, tipos de erros, Kahoot e estratégias de debugging.

2º Ciclo

Gincana sem IA: resolução manual dos desafios em pares.

Coffee Break

Pausa de 15 minutos para recarregar as energias.

3º Ciclo

Gincana com IA: uso guiado da IA para apoiar a investigação.

4º Ciclo

Roda de conversa, questionário, ranking final e encerramento.



Cada ciclo tem tempo cronometrado. Fique atento ao relógio da arena!

Duas formas de investigar bugs

Gincana sem IA

- Resolução manual dos desafios
- Discussão de hipóteses em pares
- Uso de estratégias de debugging
- Foco em raciocínio lógico e investigação pura

Gincana com IA

- IA como apoio investigativo — não como solução pronta
- Pistas, perguntas e explicações são permitidos
- O par precisa validar e entender a resposta
- Defender tecnicamente a solução encontrada

"Na Debug Arena, vence quem **investiga melhor**, não quem chuta mais rápido."

Pronto para entrar na arena?

Investigar

Trate cada bug como um mistério a ser desvendado.

Hipotetizar

Teste uma ideia de cada vez com método e rigor.

Defender

Explique sua solução com clareza e confiança técnica.

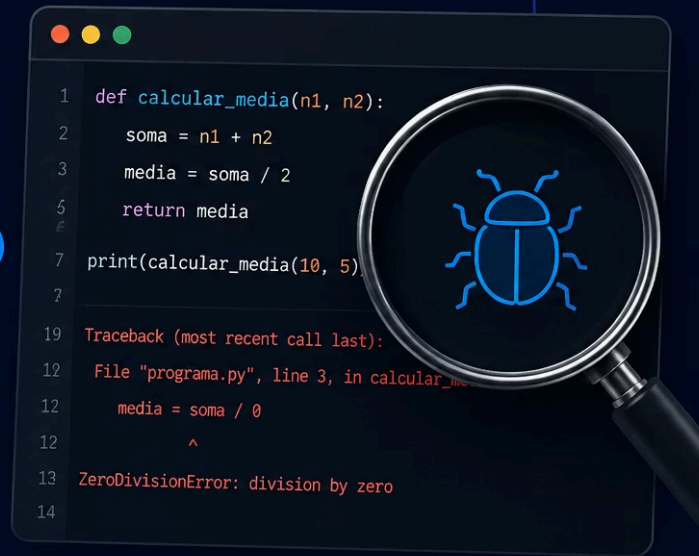
✔  A Debug Arena está aberta. Que comece a investigação!





Tipos de Erros: o primeiro passo para depurar

Antes de corrigir um **bug**,
precisamos entender o tipo do **erro**.



Nem todo erro é igual

Assim como um detetive investiga pistas diferentes, cada tipo de erro exige uma abordagem distinta. Reconhecer o tipo é o primeiro passo para resolver o problema.

🚫 Impede de rodar

O código nem começa a executar.

💥 Para no meio

O programa inicia, mas trava durante a execução.

🤔 Funciona errado

Roda sem avisar, mas entrega resultado incorreto.

🔍 **Lembre-se:** Você não resolve um mistério sem entender a pista.



● ERRO #1

Erro de Sintaxe

O código **quebra as regras da linguagem**. É como escrever uma frase sem fechar a pontuação: o Python simplesmente não entende o comando.

✗ Código com erro

```
print("Olá"
```

✓ Código corrigido

```
print("Olá")
```

💬 Mensagem do Python

```
SyntaxError: unexpected EOF  
while parsing
```

O Python apontou exatamente onde a frase ficou incompleta. A mensagem de erro é sua primeira pista!

Como reconhecer um Erro de Sintaxe?

O Python é rigoroso. Pequenos deslizes quebram o código antes mesmo de rodar.

(

Parênteses abertos

```
print("Olá"
```

66

Aspas não fechadas

```
nome = "Ana
```

:

Dois-pontos esquecidos

```
if x > 0
```

≡

Indentação incorreta

```
print("oi")
```

✎

Palavra digitada errado

```
prnt("Olá")
```

Erro de Execução (Runtime)

O programa **começa a rodar**, mas encontra um problema no caminho. O código parecia certo — mas algo deu errado durante a execução.

✗ Exemplo clássico

```
x = 10  
y = 0  
resultado = x / y
```

💬 Mensagem do Python

```
ZeroDivisionError:  
division by zero
```

Outros erros comuns



NameError

Variável que não existe



TypeError

Operação com tipos incompatíveis



IndexError

Índice fora do limite da lista

Como reconhecer um Erro de Execução?

⚡ Programa para de repente

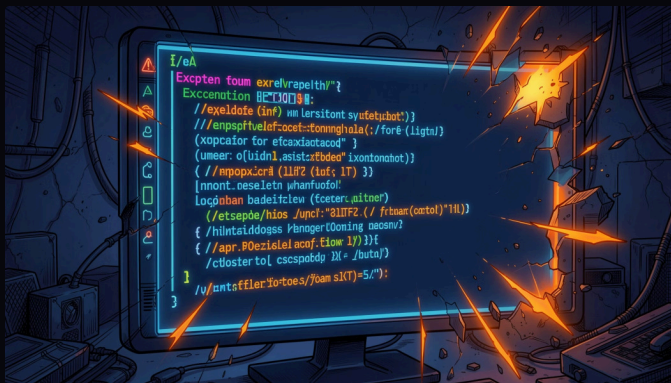
A execução é interrompida antes do fim.

📋 Terminal mostra "Traceback"

O Python exibe o caminho do erro com o nome da exceção.

🎯 Erro depende da entrada

Funciona com alguns dados, mas falha com outros.



Sinais de alerta no terminal

Observe como os sintomas aparecem juntos: a execução para, o terminal destaca o traceback e o comportamento muda conforme a entrada.

● ERRO #3

Erro de Lógica

O programa **roda sem nenhuma mensagem de erro**, mas entrega um resultado errado. O computador fez exatamente o que você mandou... não o que você queria.

✗ Código com erro lógico

```
nota1 = 8  
nota2 = 6  
media = nota1 + nota2 / 2  
print(media) # Resultado: 11.0
```


Como reconhecer um Erro de Lógica?

Este é o mais sorrateiro: **não há mensagem de erro**. O programa roda, mas o resultado não faz sentido.



Resultado estranho

O valor calculado parece errado ou inesperado.

$+ = \times$

Fórmula errada

A ordem das operações ou a expressão matemática está incorreta.



Condição invertida

Um `if` que deveria ser verdadeiro sempre retorna falso.



Repetições incorretas

Um `for` ou `while` que executa vezes demais ou de menos.



O programa rodou... mas o resultado faz sentido?

Comparando os Tipos de Erro

Cada erro tem sua "assinatura". Aprenda a identificá-la antes de agir.

Tipo de Erro	O programa roda?	Há mensagem?	Exemplo
 Sintaxe	Não	Sim, <code>SyntaxError</code>	<code>print("Olá"</code>
 Runtime	Começa e quebra	Sim, exceção	<code>10 / 0</code>
 Lógica	Sim	Nem sempre	<code>media = a + b / 2</code>

 **Sintaxe**

Regras quebradas

 **Runtime**

Falha na execução

 **Lógica**

Resultado incorreto

Agora é sua vez: que tipo de erro é esse?

No **Kahoot!**, você vai analisar exemplos reais e classificar cada bug. Lembre-se das pistas de cada tipo!

● Sintaxe

O código não roda. Procure por parênteses, aspas e dois-pontos.

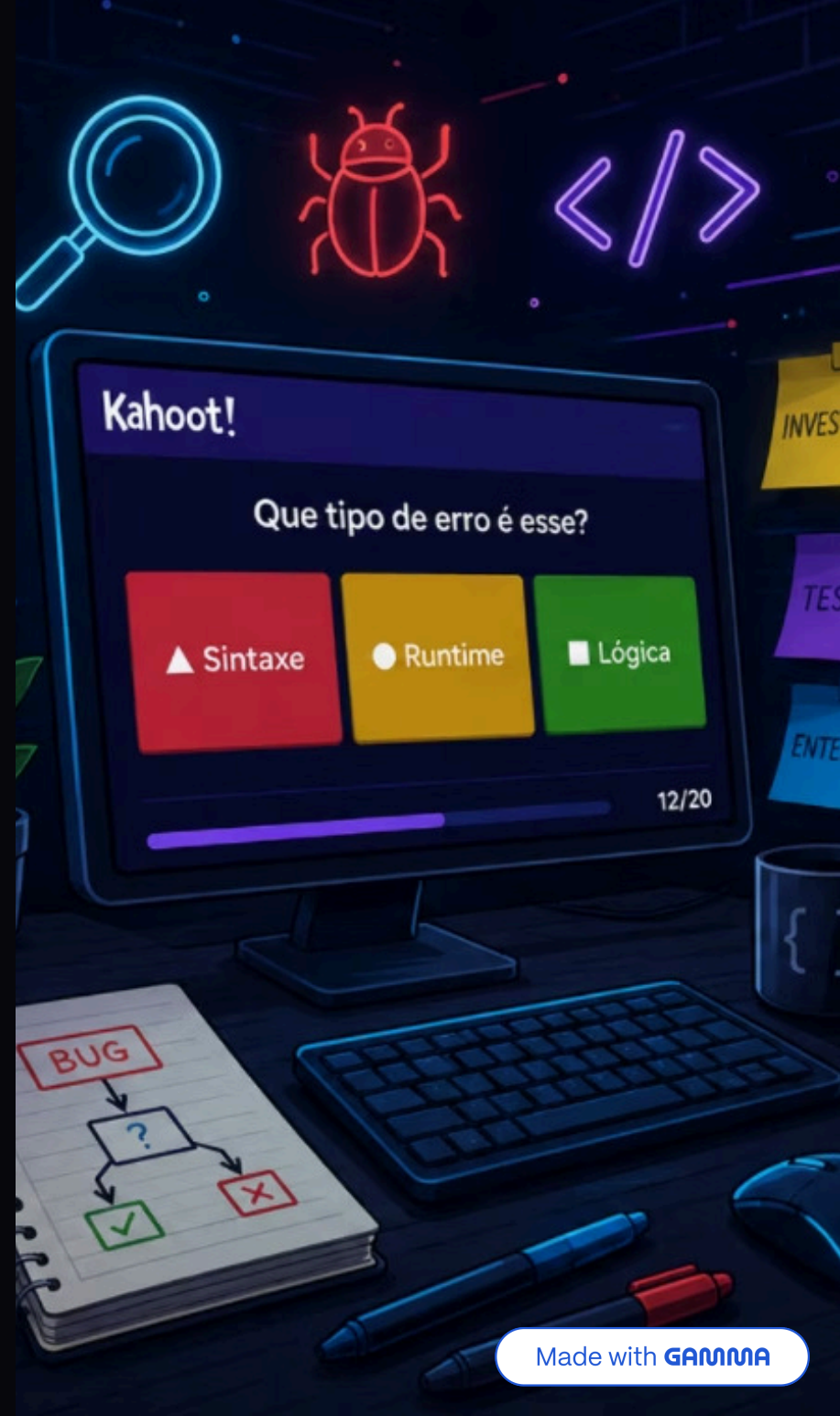
● Runtime

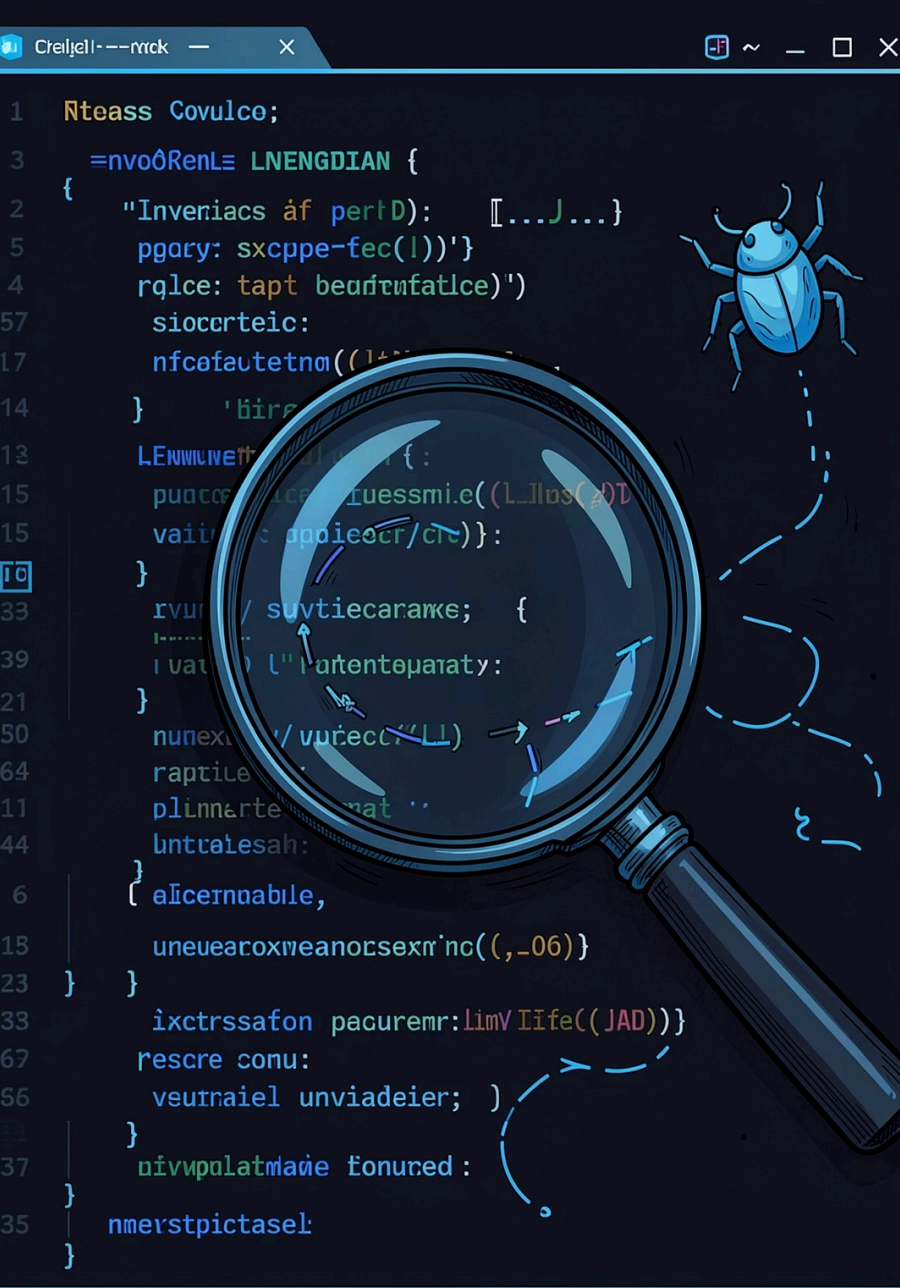
O programa para no meio. Procure por Traceback e exceções.

● Lógica

Roda, mas o resultado está errado. Questione a lógica do código.

✅ 🏆 Na Debug Arena, cada erro é uma pista. O desafio é **investigar antes de corrigir**.





Estratégias de Debugging

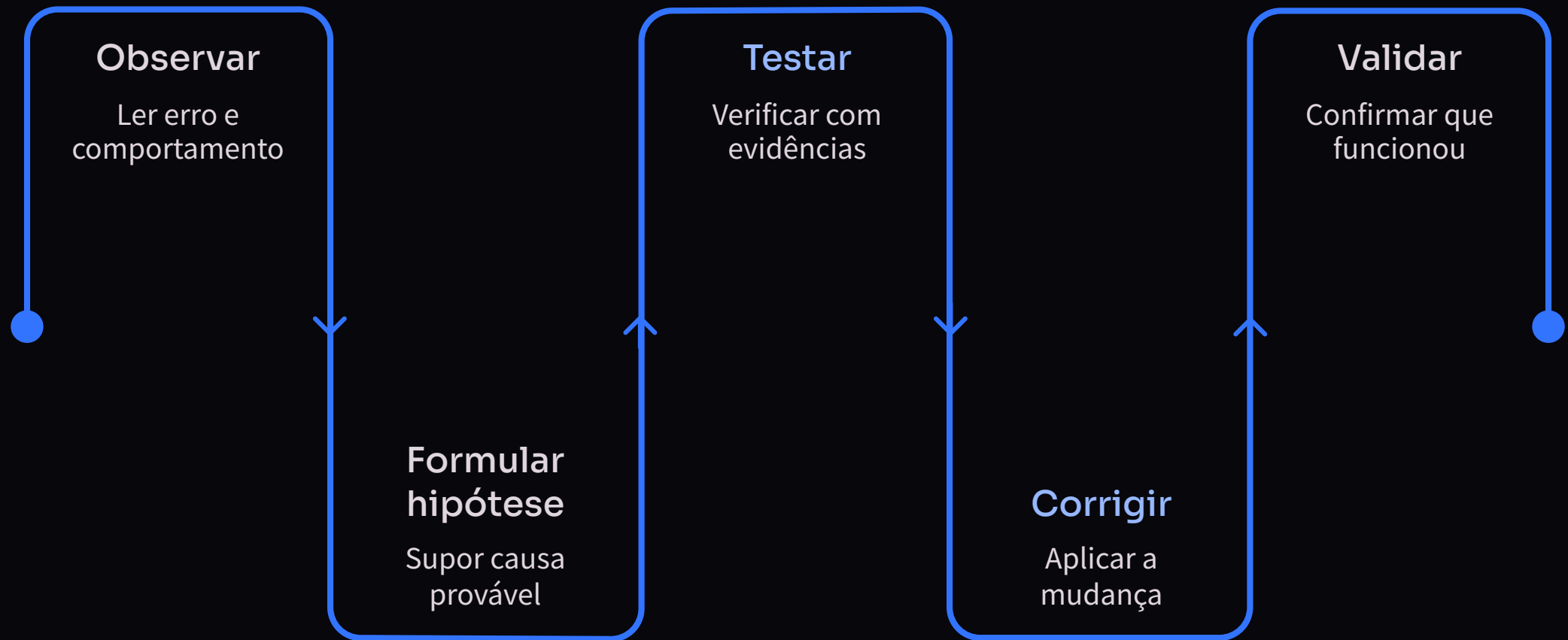
Investigando antes de corrigir

Depurar não é adivinhar. É seguir pistas, testar hipóteses e validar soluções.



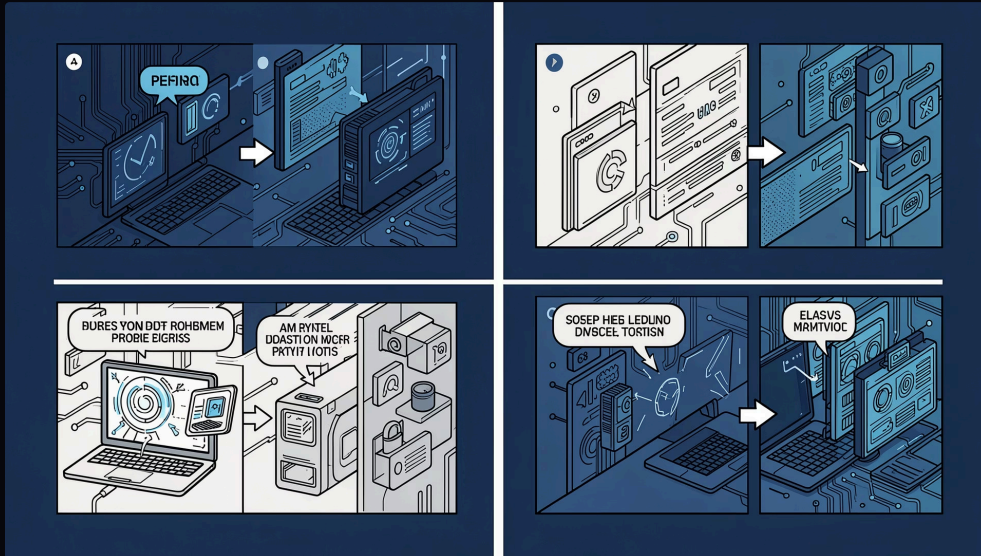
Debugging não é tentativa e erro

Corrigir bugs sem método pode gerar mais confusão. **Antes de mudar o código, entenda o problema.**



Seguir esse fluxo transforma o debugging em investigação — não em chute.

Divide and Conquer



O que é?

Dividir o problema em partes menores para descobrir **onde exatamente a falha acontece**.

  Se o problema parece grande, divida em partes menores.

Fluxo do programa



Como aplicar Divide and Conquer?

1

Identifique o erro

Onde o comportamento inesperado aparece?

2

Separe em blocos

Divida o código em trechos menores e independentes.

3

Teste uma parte por vez

Isolando blocos, você descobre qual está errado.

4

Encontre o trecho culpado

O comportamento muda? O erro está aqui.

```
def calcular_media(notas):  
    soma = sum(notas) # Bloco 1 ✓  
    media = soma / len(notas) # Bloco 2 ← erro aqui?  
    return media
```


Print Statements

Como funciona?

Inserir `print()` temporários para **observar valores e caminhos** do programa durante a execução.

 Prints funcionam como lanternas dentro do código.

Exemplo em Python

```
x = 5
print("valor de x:", x)

if x > 3:
    print("entrou no if")
    y = x * 2
    print("y =", y)
```

Saída: valor de x: 5 → entrou no if → y = 10

ESTRATÉGIA 3

Testes de Hipóteses

Criar uma suposição sobre a causa do erro e testá-la de forma controlada, sem mudar o código aleatoriamente.

🤔 Hipótese

"O erro acontece porque a variável está vazia."

🧪 Teste

"Vou imprimir o valor antes do cálculo."

✅ Resultado

A hipótese foi confirmada ou descartada.

📄 Uma boa hipótese evita mudanças aleatórias no código.

Tracing – Execução passo a passo

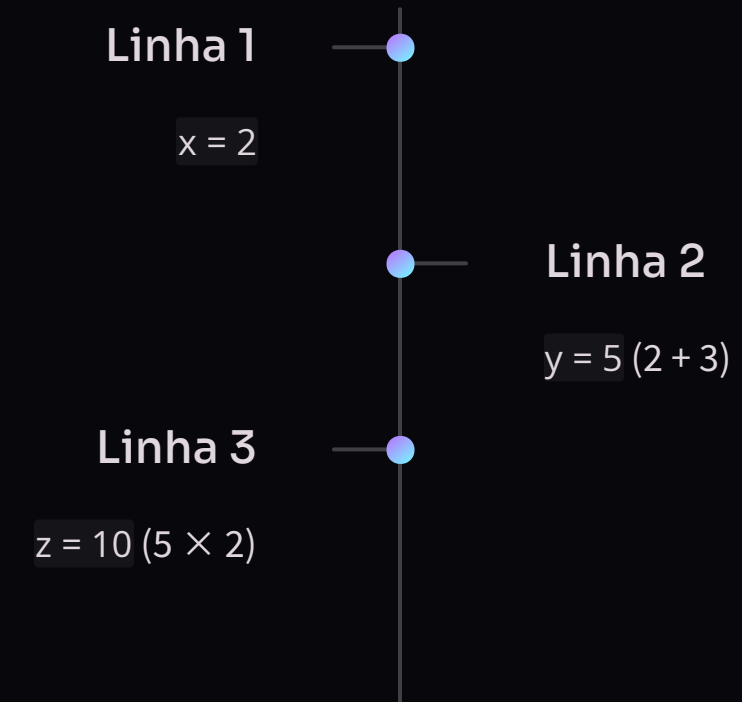
O que é?

Acompanhar o código **linha por linha**, registrando como os valores das variáveis mudam a cada passo.

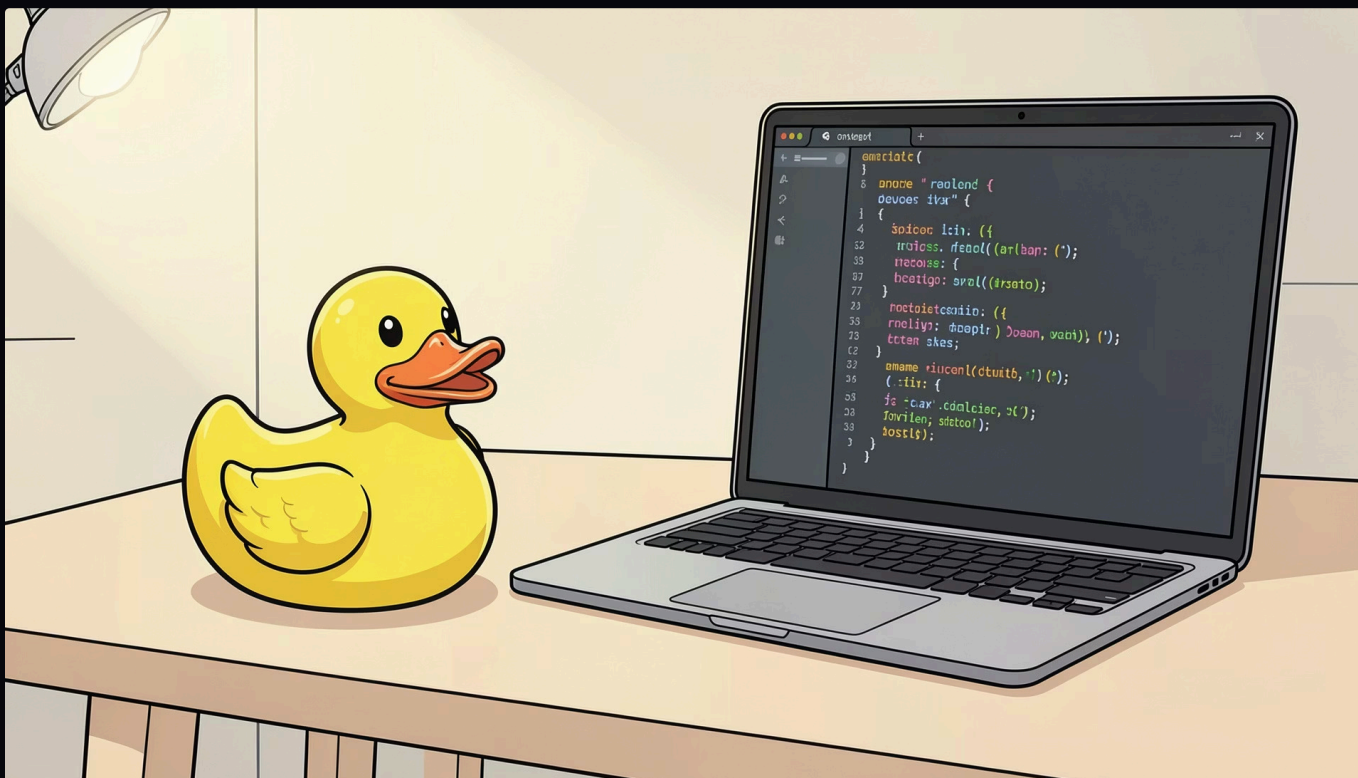
🔍 Às vezes o bug não está no resultado final, mas no caminho.

Exemplo: rastreando valores

```
x = 2  
y = x + 3  
z = y * 2
```



Rubber Duck Debugging



Como funciona?

Explicar o código **em voz alta**, para um colega, um objeto ou até para você mesmo, ajuda a perceber falhas que estavam passando despercebidas.

"Esta linha deveria calcular a média... mas por que a divisão acontece *antes* da soma?"



 Se você consegue explicar, consegue investigar melhor.



ESTRATÉGIA 6

Code Review Colaborativo

Revisar o código com outra pessoa ajuda a encontrar erros e validar hipóteses com mais rapidez.

Piloto

Manipula o código, digita e executa os testes.

Navegador

Observa, questiona e sugere caminhos sem tocar no teclado.

-  Explicar o código para alguém ajuda a enxergar o erro com mais clareza.

Qual estratégia usar?

Cada situação pede uma abordagem diferente. Use este guia rápido:

Situação	Estratégia
Problema grande ou confuso	Divide and Conquer
Não sei o valor das variáveis	Print Statements
Tenho uma suspeita sobre a causa	Testes de Hipóteses
Preciso entender o caminho do programa	Tracing
Não consigo enxergar o erro sozinho	Rubber Duck
Preciso validar minha solução	Code Review Colaborativo